

Informe Técnico

Segundo Proyecto Largo

Juan Morales C.I.: 30916541

Omaira Rodriguez C.I.: 15351394

Keiber Mendina C.I.: 31401788

Análisis de Sistemas

Lenguajes de Programación(C7)

1. Descripción General

El presente proyecto consiste en el diseño e implementación de un software desarrollado en Rust capaz de transformar expresiones aritméticas en notación infija a sus equivalentes en notaciones postfija (Notación Polaca Inversa o RPN) y prefija (Notación Polaca).

El núcleo del sistema destaca por las siguientes características de diseño:

- **Persistencia de datos:** La expresión y su correspondiente tabla de símbolos estructurada son leídas dinámicamente desde un archivo de texto plano.
- **Seguridad y Control de Memoria:** Fiel a la filosofía de Rust, el análisis sintáctico y la tokenización se ejecutan exclusivamente mediante arreglos de tamaño fijo, evitando la asignación dinámica de memoria en tiempo de ejecución (vectores dinámicos) y mitigando riesgos de desbordamiento de memoria de manera manual y explícita.

2. Estructuras de Datos

Para prescindir del uso de memoria dinámica y optimizar el rendimiento, el programa utiliza la técnica de arreglos paralelos coordinados mediante un índice de control de longitud (usize). A continuación, se detallan los componentes estructurados del software:

2.1. Tabla de Símbolos

Almacena la configuración de los caracteres válidos leídos desde el archivo de configuración.

Componente	Tipo de Datos	Capacidad Máxima	Descripción
simbolos	[char; MAX_SIMBOLOS]	50	Almacena el carácter literal del símbolo.
tipos	[Tipo; MAX_SIMBOLOS]	50	Determina si el token es OPERANDO u OPERADOR.
precedencias	[i32; MAX_SIMBOLOS]	50	Nivel jerárquico del símbolo para las operaciones.
cantidad	usize	Variable	Lleva el control de cuántos símbolos se han registrado.

2.2. Tokens de la Expresión

Estructura encargada de alojar la cadena procesada y descompuesta individualmente después del proceso de tokenización.

Componente	Tipo de Datos	Capacidad Máxima	Descripción
simbolos	[char; MAX_EXPRESION]	100	Caracteres ordenados de la expresión.
tipos	[Tipo; MAX_EXPRESION]	100	Naturaleza gramatical de cada token.
precedencias	[i32; MAX_EXPRESION]	100	Jerarquía asociada para el algoritmo de ordenamiento.
cantidad	usize	Variable	Número de tokens válidos en la expresión actual.

2.3. Pila de Operadores

Estructura LIFO (*Last In, First Out*) utilizada de manera interna por los algoritmos de conversión.

Componente	Tipo de Datos	Capacidad Máxima	Descripción
pila_simbolos	[char; 100]	100	Contenedor físico de los operadores en espera.
pila_precedencias	[i32; 100]	100	Prioridades de los operadores en la pila.
tope	usize	Variable	Puntero de control del elemento superior de la pila.

3. Algoritmos Implementados

El comportamiento lógico para resolver las expresiones sigue las reglas de la teoría de autómatas y compiladores.

3.1. Notación Infija a Postfija (Algoritmo de Shunting Yard)

Ideado por Edsger Dijkstra, este método procesa los tokens de izquierda a derecha apoyándose en la pila para retener los operadores según su prioridad.

Para cada token en la expresión infija:

Si token es OPERANDO:

Agregar token al arreglo postfijo

Si token es OPERADOR:

Mientras la pila no esté vacía

Y precedencia(pila[tope]) $\geq$ precedencia(token):

Sacar el tope de la pila

Agregar el tope extraído al arreglo postfijo

Meter el token actual en la pila

Al finalizar el recorrido:

Mientras la pila no esté vacía:

Sacar el tope y agregarlo al arreglo postfijo

3.2. Notación Infija a Prefija

La conversión a prefija se logra adaptando el algoritmo base de Shunting Yard mediante transformaciones sobre la cadena:

1. Invertir el orden original de los tokens de la expresión infija.

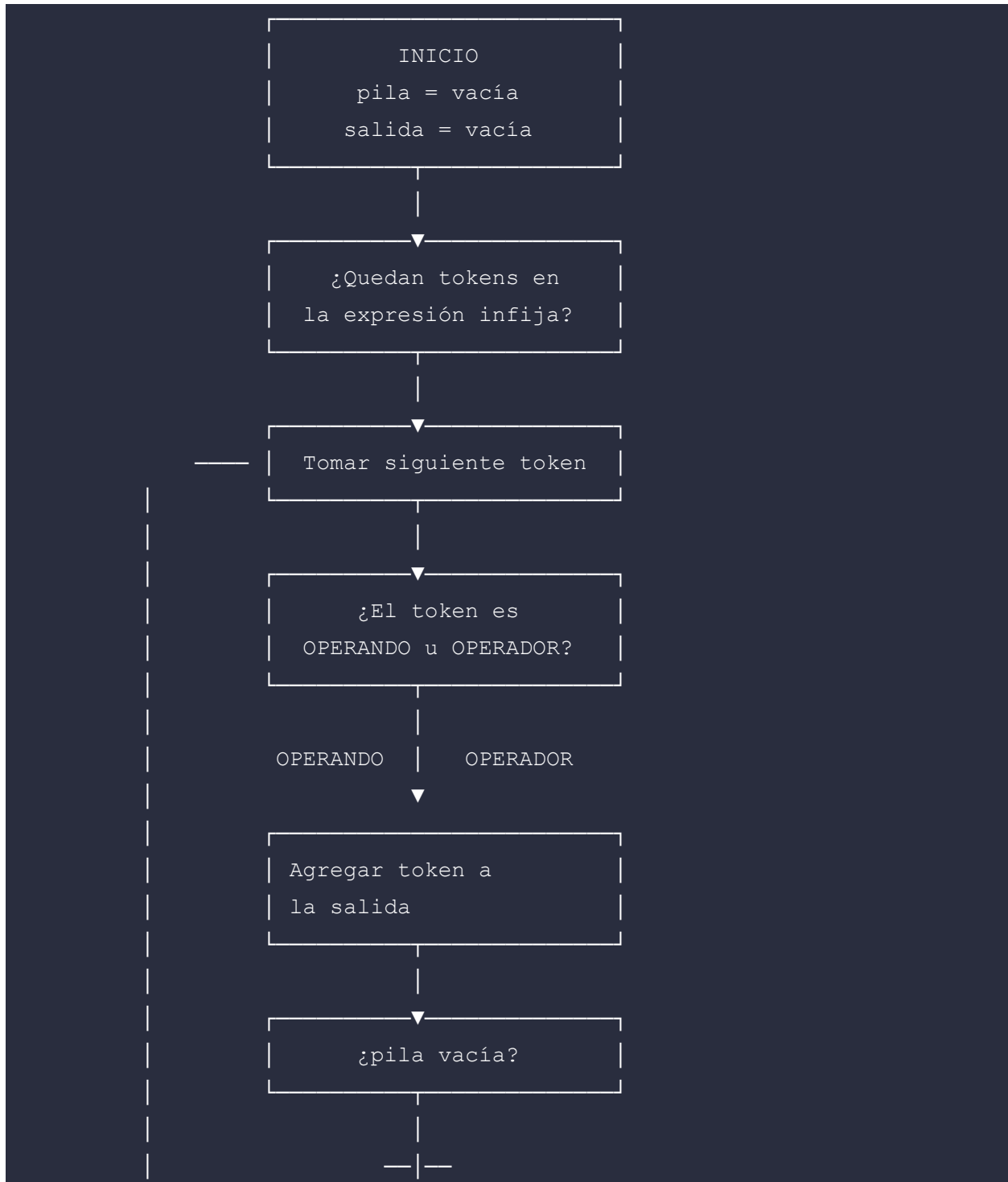
2. Aplicar el algoritmo de Shunting Yard con una variación en la restricción de precedencia:

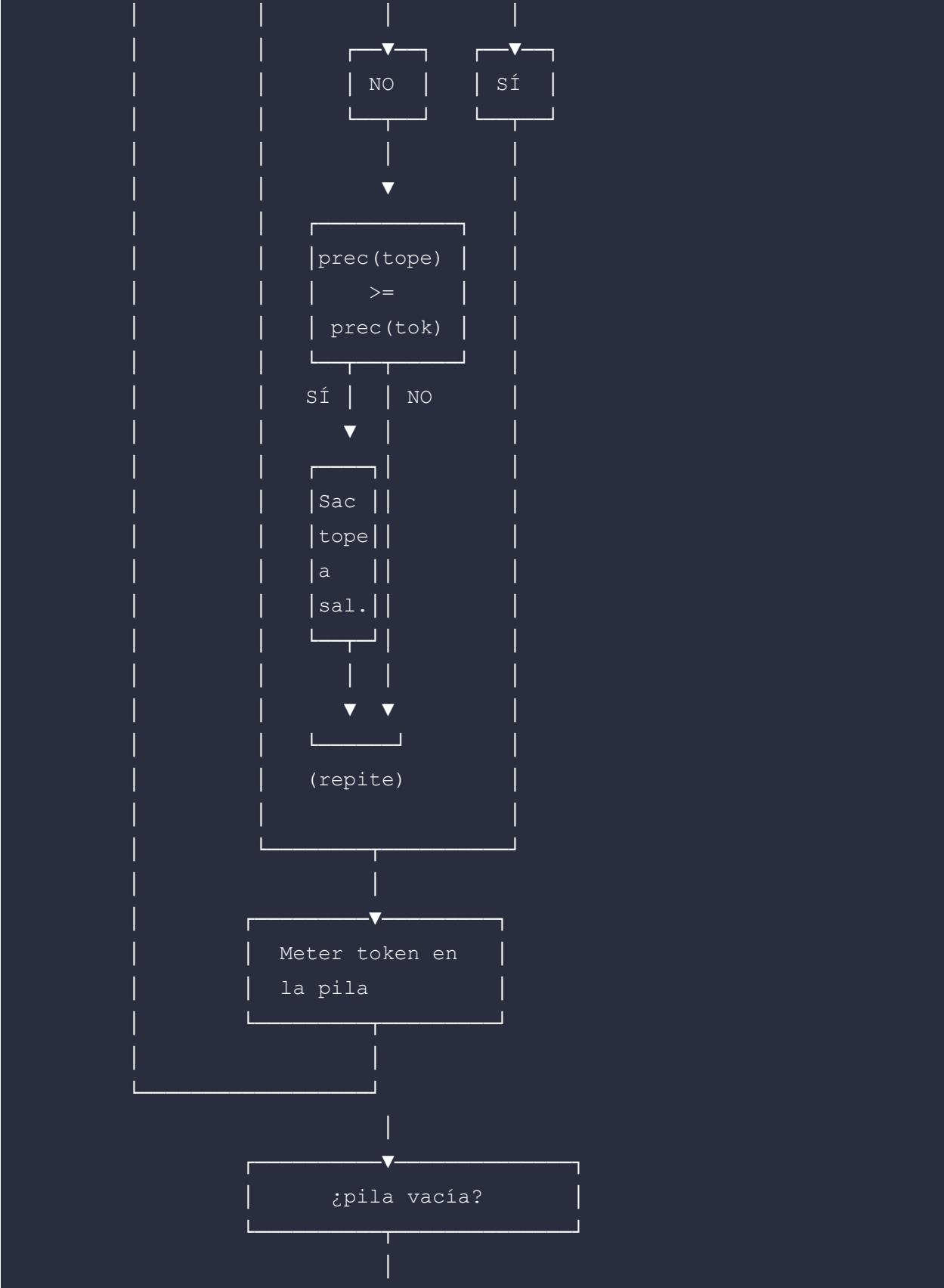
Se utiliza una comparación estricta ($>$ en lugar de $\geq$) al evaluar el tope de la pila.

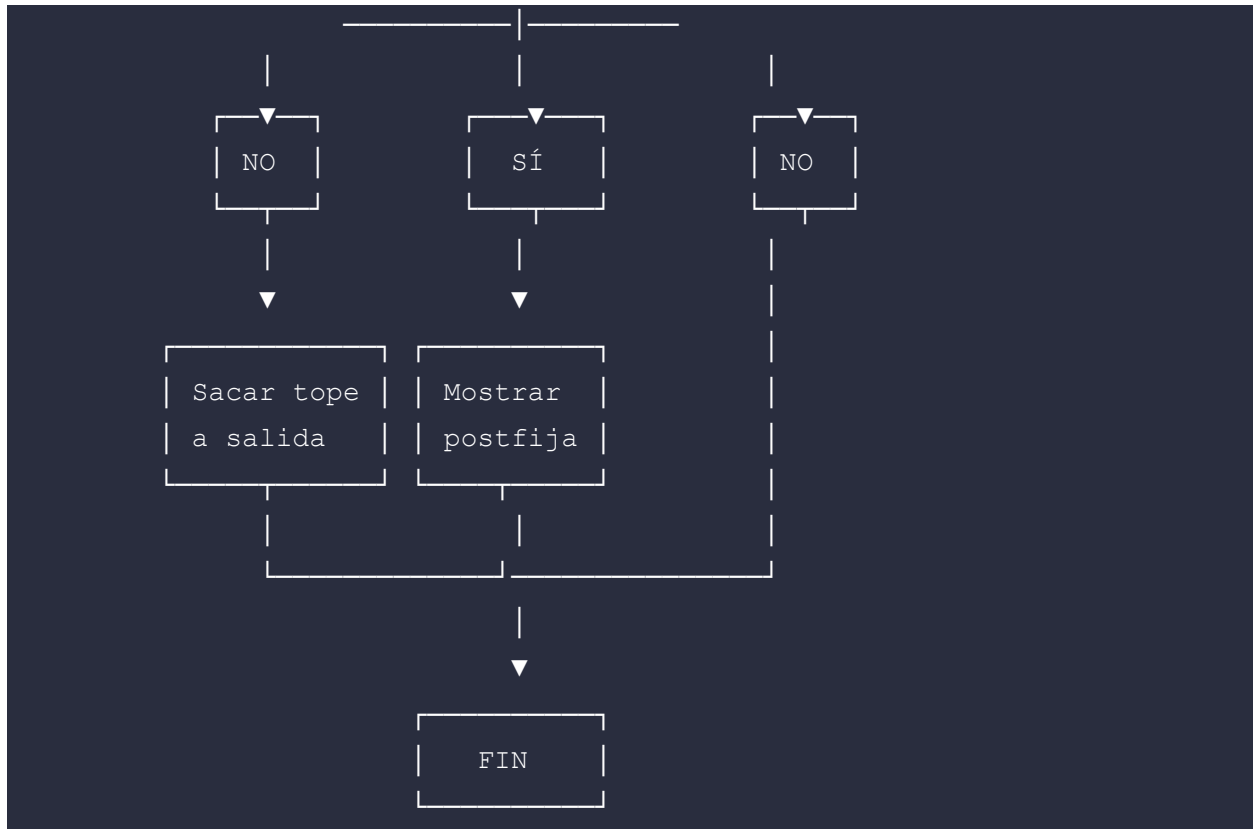
3. Invertir el arreglo resultante para obtener la secuencia prefija correcta.

4. Diagrama de Flujo del Proceso de Conversión (Postfija)

El flujo sistemático que sigue la lógica de conversión se ilustra en el siguiente esquema conceptual:







5. Módulos y Funciones de la Arquitectura

El código se encuentra modularizado bajo funciones específicas y de responsabilidad única:

- **leer_archivo:** Gestiona el flujo de entrada/salida (I/O). Se encarga de abrir el archivo físico, procesar el texto línea por línea y parsear la información hacia las estructuras de memoria correspondientes.
- **buscar_simbolo:** Actúa como función auxiliar de búsqueda. Compara un carácter específico contra los registros existentes en la tabla de símbolos para validar su existencia y retornar sus propiedades.
- **tokenizar:** Toma la cadena raw de la expresión aritmética y la descompone de forma estructurada en el arreglo paralelo de tokens primarios.
- **mostrar_tabla_tokens:** Función utilitaria de depuración y salida que imprime en la terminal los tokens detectados con sus respectivos tipos y precedencias con un formato legible.
- **infija_a_postfija:** Implementa la lógica del Shunting Yard clásico para resolver la notación postfija.
- **infija_a_prefija:** Orquesta los giros de cadena y las variaciones de prioridad lógica para producir la salida en notación prefija.

6. Control de Errores y Casos Borde

Para garantizar un software robusto y alineado a las buenas prácticas de Rust, se diseñó un sistema de control de excepciones manual para los siguientes escenarios críticos:

1. Gestión de Archivos:

- **Archivo no encontrado:** El programa captura el fallo de apertura del sistema de archivos, despliega un mensaje informativo claro para el usuario y aborta de forma segura con un código de salida 1.
- **Archivo vacío:** Valida la existencia de la cabecera e interrumpe la ejecución si el documento carece de datos legibles.

2. Validación de Sintaxis en Configuración:

- **Formato inválido:** Controla que cada línea de definición de la tabla de símbolos contenga exactamente 3 campos delimitados por comas.
- **Tipo desconocido / Precedencia inválida:** Rechaza palabras clave ajenas a OPERANDO u OPERADOR, y obliga a que las jerarquías de prioridad se resuelvan como números enteros (i32) válidos.

3. Análisis de la Expresión:

- **Expresión vacía:** El sistema cancela el proceso si la instrucción EXPRESION= no contiene caracteres subsiguientes.
- **Símbolo no definido:** Si durante la tokenización un carácter no se encuentra mapeado en la tabla de símbolos previa, se dispara un error de compilación/análisis sintáctico.

4. Seguridad de la Memoria (Desbordamiento):

- Debido al uso estricto de arreglos estáticos limitados por constantes (MAX_SIMBOLOS = 50 y MAX_EXPRESION = 100), se añadieron guardas lógicas antes de cada inserción. Si la expresión supera los 100 caracteres o la tabla los 50 elementos, el programa interrumpe el flujo impidiendo cualquier intento de *buffer overflow*.